

Inflation Attack Stateful Fuzzer

Overview

This project is a generic ABI-driven stateful fuzzer built for discovering inflation attack vulnerabilities in DeFi vault-style smart contracts.

The main goal of the project is to automatically discover transaction sequences where a vault's accounting is manipulated using direct asset donations, causing a victim to deposit assets but receive zero shares due to manipulated share pricing and integer division rounding.

Instead of hardcoding a fixed exploit flow, the fuzzer dynamically discovers protocol functionality from the contract ABI and explores protocol state transitions using randomized multi-transaction sequences.

What Problem Does This Project Solve?

Many DeFi vault protocols mint shares using a ratio based on total assets and total supply. If an attacker manipulates vault assets using direct token donations, future deposits may mint zero shares due to rounding behavior.

This creates an inflation attack where victims lose assets while receiving no ownership shares.

What Makes This Fuzzer Generic?

The fuzzer is not hardcoded to only execute a fixed exploit path.

Instead, it dynamically:

- loads the ABI
- discovers callable functions
- generates transaction sequences automatically
- mutates sequences dynamically
- executes sequences against evolving blockchain state

This makes the engine protocol-driven instead of exploit-script-driven.

Project Structure

inflation-fuzzer/

|

|—— contracts/

| |—— MockERC20.sol

| |—— VulnerableVault.sol

| |—— interfaces/

| |—— IVaultLike.sol

|

|—— script/

| |—— Deploy.s.sol

|

|—— fuzzer/

| |—— bootstrap.py

| |—— config.py

| |—— coverage.py

| |—— engine.py

| |—— executor.py

| |—— minimizer.py

| |—— oracle.py

| |—— sequences.py

| |—— state.py

|

|—— test/

| |—— Vault.t.sol

|

|—— foundry.toml

|—— README.md

Solidity Contracts

MockERC20.sol:

A simplified ERC20 implementation supporting mint, transfer, approve, and transferFrom.

VulnerableVault.sol:

A deliberately vulnerable vault contract whose accounting can be manipulated using direct token donations.

IVaultLike.sol:

An interface standardizing vault interaction methods used by the fuzzer.

Python Fuzzer Architecture

The Python fuzzer handles:

- ABI discovery
- transaction generation
- state mutation
- multi-step execution
- exploit detection
- exploit minimization

Control Flow

1. Deploy contracts locally using Foundry.
2. Start Anvil local blockchain.
3. Load ABI artifacts dynamically.
4. Discover callable functions.
5. Generate randomized transaction sequences.
6. Execute sequences against evolving blockchain state.
7. Detect inflation anomalies.
8. Minimize exploit traces.
9. Save exploit.json output.

How To Run The Project

Step 1 — Install Git

Download Git from: <https://git-scm.com/downloads>

Step 2 — Install Python

Download Python from: <https://www.python.org/downloads/>

Enable 'Add Python to PATH' during installation.

Step 3 — Install Foundry

Run:

```
iwr https://raw.githubusercontent.com/foundry-rs/foundry/master/foundryup/install.ps1 -useb | iex
```

Then run:
foundryup

Step 4 — Install forge-std
git init
forge install foundry-rs/forge-std

Step 5 — Install Python Dependencies
pip install web3

Step 6 — Build Contracts
forge build

Step 7 — Start Anvil
anvil

Step 8 — Deploy Contracts
forge script script/Deploy.s.sol --broadcast --rpc-url http://127.0.0.1:8545 --private-key
YOUR_PRIVATE_KEY

Step 9 — Run Tests
forge test

Step 10 — Run Fuzzer
cd fuzzer
python engine.py

Final Notes

This project helped me understand:

- DeFi vault accounting
- ERC4626-style share logic
- integer division vulnerabilities
- stateful fuzzing
- ABI-driven exploration
- exploit minimization
- blockchain testing infrastructure